Transit-Node Site Screening — how it works

Input · Python pipeline · LLM / agent · Output

Site candidates + store concept
3 Berlin transit nodes · all-day convenience store

Open public data
OSM, Overture, LOR, IHK, Wikipedia, GTFS

fetch

Collect & clean
fetch, dedupe, merge POIs across providers;
store as dated SQLite snapshots

suspect records

LLM data audit · Claude
Rules list records that look wrong;
Claude checks each one with web search
Code guard → an analyst accepts or rejects
Never changes a number itself

accepted fixes

Metrics
people per day (residents + workers + visitors);
competitors open per day

refetch → recompute

Data gap that could change the result?

yes

Follow-up agent
Policy: A rule / B impact-ordered (default) / C Claude
Code guard: allowed tools, max 4 calls per run
Action: refetch data, or flag a field check

no / budget spent

Scoring
business fit and evidence quality,
scored separately

scales

139 reference stations
P10 / P90 anchors for demand and competition

Sensitivity analysis
inputs first (visitors, workers, under-mapping,
both at once), then weights

Outputs
sourced report: diligence ranking,
evidence gaps, field-research checklist